JEPA, and where Felix may learn its choices

docs/2026-10-19-jepa-and-learned-choices.md · built Mon 19 Oct 2026

Research note, 2026-10-19. The founder asked two questions that day:

- whether JEPA models can apply to Felix's recursive improvement;
- whether "deterministic only" should stay an absolute rule for the half of Felix outside the trust plane, once Felix runs at scale.

Standing. Research, not direction. Nothing here is a build target, and nothing in `docs/direction.md` §5 moves. The two checks in §6 and the field in §7 are proposals for when §7 of the intent-to-build design and lane item 7 (the outcome record) are built.

1. The answer

- **JEPA as a part of the loop: no.** The loop has almost nothing to learn from, its deciding steps are the enforcing path, and a learned dimension cannot be admitted by a person. Felix also already predicts in an abstract space, which is JEPA's main idea, with an encoder it designed by hand on purpose (§4).
- **JEPA as a source of design: mostly a confirmation.** JEPA's central problem is collapse, where the predictor and the thing it predicts agree trivially. Every guard the field found against it has a counterpart already written into the intent design's §7, except one. That field reached the same structure on its own, which is worth knowing (§5).
- **What it adds:** two zero-token checks for the learned tables, a redundancy count and a collapse report (§6).
- **On determinism: not an absolute no, and the line is not between the planes.** It runs between *binding* and *choosing*. A decision that binds stays deterministic in either plane. A choice among options the frame has already admitted may be learned, and even randomized, once there are outcomes to learn from. Learn how to choose; never learn what binds (§7).
- **The one thing worth doing early:** when the outcome record is specified, give each choice a record of the options it had and the probability the chosen one had. Without that, no later learning can score an alternative that was never tried (§7).

2. What JEPA is

A Joint-Embedding Predictive Architecture has two encoders and a predictor. One encoder reads a context (part of an image, the past frames of a video), and the other reads a target (the masked part, the future frames). The predictor maps the context's representation to a guess at the target's representation. The loss is the distance between the two representations, so nothing is ever reconstructed pixel by pixel. The point is to spend no capacity on detail that cannot be predicted. LeCun proposed it in 2022, and I-JEPA (images, 2023) and V-JEPA (video, 2024) followed.

Collapse is the central failure. The loss is zero if both encoders output the same constant for every input. Three families of fix exist:

- **Asymmetry.** The target encoder is a slow moving average of the context encoder, and the loss does not train it directly (stop-gradient). I-JEPA and V-JEPA do this.
- **Variance and covariance terms.** VICReg-style terms require each dimension to vary across a batch and penalise dimensions that duplicate each other.
- **A declared target distribution.** LeJEPA (Balestriero and LeCun, November 2025) regularises embeddings toward an isotropic Gaussian with one term, SIGReg, and the authors prove this excludes collapse without the teacher networks and schedules the earlier methods needed.

World models and planning. V-JEPA 2 (Meta, June 2025) pre-trained a video encoder on more than a million hours of video. A small action-conditioned predictor, V-JEPA 2-AC, was then post-trained on under 62 hours of robot video. It plans by model-predictive control: it searches action sequences (with the cross-entropy method) for the one whose predicted representation lands closest to a goal image's, executes only the first action, observes, and plans again. It ran zero-shot on Franka arms in two labs. The demonstrated tasks are short-horizon (reach, pick, place), and goals are images, not language. A December 2025 study of what makes these planners work beat both DINO-WM and V-JEPA 2-AC on navigation and manipulation.

Language. LLM-JEPA (Huang, LeCun and Balestriero; ICLR 2026) adds a JEPA term to next-token training, for example predicting the embedding of a code snippet from the embedding of its description. It improved fine-tuning on NL-to-regex, Spider (NL-to-SQL), GSM8K and others. Its stated limits are that it needs two genuine views of the same content, roughly doubles training compute, and adds two hyperparameters that are costly to tune.

The cost module. In LeCun's 2022 architecture, the cost has two parts. The *intrinsic cost* is hard-wired and not trainable. The *critic* is trainable and learns to predict future intrinsic cost. The world model and actor learn to reduce the cost; nothing learns to change it.

Where it stands. LeCun left Meta in November 2025, and AMI Labs launched in March 2026 with a $1.03 billion seed round. It is aimed at robotics, manufacturing, industrial control and healthcare. JEPA papers in 2026 span video, driving, medical imaging, time series and genomics. **No JEPA for code or software engineering turned up in the search.** The nearest "world model for code" is Meta FAIR's CWM (September 2025, 32B). It was mid-trained on Python execution traces and agent trajectories, scores 53.9 on SWE-bench Verified, and is an ordinary autoregressive model, not a JEPA.

3. Felix's recursive improvement, as designed

It is §7 of `docs/2026-10-19-intent-to-build-design.md`, a design on file. Nothing in it is built.

1. A miss is labelled only by a mechanical join: a revert, a reopened obligation, a failed post-deploy probe, a person-channel incident, or the sealed corpus.
2. A fixed walk classes the miss into one of eight gap kinds, or prints it as unclassifiable.
3. A model drafts one class-level row.
4. The engine checks the row: vocabulary only, prevention at the miss tree, blind sibling fixtures, exposure, and identical bytes on two offline runs.
5. The row enters with no force.

6. Force comes from the founder. After control 7 and the judge pin, task-blocking force may also come from thresholds fixed in advance.
7. The machinery itself is never earnable.

Today the loop has little to run on:

- The outcome record (lane item 7) is not built.
- The land ledger holds 54 rows and 2 lander merges, #451 and #454, both on 10-19.
- 0 of 22 proposed obligations are admitted.
- Main has 633 commits.

4. Why JEPA cannot be a part of the loop

1. **There is nothing to train on.** JEPA's results rest on volume. Even the small robot stage of V-JEPA 2 used 62 hours of video, after a million hours of pre-training. Self-supervision removes the need for labels, not the need for data. Felix's loop learns from recorded misses and lagged outcomes. It has two unattended merges and no outcome rows.
2. **The loop's deciding steps are the enforcing path.** Triage, validation and promotion decide what may bind. The constitution's invariant 2 says the enforcing path is bash and git only, and direction row 6 adds no new runtime there. A frozen network gives the same answer every time, but it is still a model on the path, and a stranger cannot replay its verdict from the repository and a receipt bundle.
3. **A learned dimension cannot be admitted.** §7's vocabulary lint lets a row use only capability tags, roles, axes and need tokens. A founder can admit "an authenticated public write owes a write ceiling". Nobody can admit dimension 417 of an embedding.
4. **Felix already predicts in an abstract space.** Probes, facts and situations make up an encoder built by hand, and a table row is a prediction stated in its vocabulary. JEPA's main lesson, to predict in representation space rather than input space, is how §7 is already built. What JEPA would add is learning the encoder from data, which is the one thing Felix's guarantees rule out.

On the proposing side, where models are allowed, a JEPA would have to draft better rows from a miss record than the frontier model Felix already rents. No JEPA for code exists to try. LLM-JEPA is a training objective for model vendors, which makes it part of the host's "how" (D1).

5. What JEPA confirms in §7

JEPA mechanism	Why JEPA needs it	Counterpart in the intent design's §7	Status
Collapse	A constant output gives zero loss and no information.	A row that fires on every tree "prevents" every miss (the ratchet). A row that fires on none is vacuous. "Prevention, not truth" alone would admit the first.	Named as risks
Stop-gradient and a slow target	Without asymmetry, a predictor and the judge it trains against collapse together.	The base engine judges a change's tables, the judge is pinned outside the change it judges (step 15), and force waits on a lag.	Specified; the order (rows 13, 25, 27 bound 4) puts the pin before any force

JEPA mechanism	Why JEPA needs it	Counterpart in the intent design's §7	Status
Variance term	Every dimension must vary across a batch.	Blind fixtures: at least two positive and two negative siblings per ecosystem, plus the exposure estimate. A row that fires on all fixtures or none fails.	Specified
Covariance term	Dimensions must not duplicate one another.	Nothing.	**New**, §6
LeJEPA's single declared target	One stated goal beats a stack of heuristics.	The ratchet guard makes the false-block rate the one printed, bounded figure.	Specified
Fixed intrinsic cost, trainable critic	Learning must never change what counts as cost.	The machinery joins the trust base and is never earnable; tables learn but only narrow.	Specified
Act one step, observe, plan again	Long open-loop plans drift.	The unit set is recomputed on read and never stored as authority, and facts are re-projected after each unit (pipeline stages 10 and 12).	Specified
Short horizons only	V-JEPA 2-AC's tasks are short; long-horizon planning is open.	The host owns the method inside a unit. Felix plans nothing with a learned model.	Consistent

JEPA's evidence on the second row is the strongest support from outside Felix for an order already decided: a self-improving system whose judge co-adapts with what it judges collapses. Pin first, then force.

6. Two checks to carry into §7 when it is built

Both read only the replay archive, cost no tokens, and change nothing that binds.

1. **Redundancy (the covariance term).** For each learned row, take its activation vector over the recorded trees and fixtures in the replay. Rows with identical vectors carry one bit between them. A row whose vector is contained in another's adds no catch the other would miss.
   - Print distinct activation patterns beside the row count.
   - Batch redundant rows together in the founder's admission return.
   - Without this, "rows learned" overstates what Felix has learned, and one miss hands catches to several rows that are really one.
2. **Collapse report for the table.** Print the share of replayed trees on which any learned row fires, in the weekly return. A share moving toward all trees is the ratchet starting. A share moving toward none means the rows are vacuous. The false-block rate shows neither until rows carry force, and rows enter with none, so this is the earlier signal.

7. Where Felix may use nondeterminism

The premise, checked. Felix's content side is already nondeterministic by design. Models write the readings, candidates, architectures, code, checkers and row drafts (intent design §1). The deck's slogan, which the thesis extends to capability, is "Do not make the AI deterministic. Make the boundaries deterministic." What is deterministic is every decision Felix itself makes. That includes the capability plane's boundary: what a project owes, the lease, the assurance floor, who may write the check, and retirement (thesis §3). So the real question is whether Felix's *own* decisions in the capability plane may ever be learned or randomized.

The answer: yes, for one class of decision, at scale. The line does not run between the planes. The capability plane has binding decisions too: the lease width (I3), the floor (I4) and the independence rule (I5). It runs between two kinds of decision:

- **Binding.** A wrong answer blocks, releases, widens authority, lowers assurance or commits to something irreversible. This stays deterministic in either plane.
- **Choosing.** The pick is among options the deterministic frame has already admitted, something the chooser cannot influence judges the result, and a wrong pick costs only time or tokens.

For choosing, a learned policy beats a fixed rule, and a little recorded randomness beats a greedy one. A fixed rule never tries the option it does not already favour, so it can lock in an early winner forever. I6's retention rule has this property: it keeps an incumbent while it is justified, and no step tries a rival.

Choosing decisions Felix owns, or will:

Decision	Today	What learning would use
Which eligible author writes a checker first (Claude or Codex, inside I5's independence rule)	Not built	Qualification pass rate and lagged survival
Which discovery strategy gets the next slice of budget	Not before March	Corroborated catches per dollar
Which recorded miss is triaged first	Not built	Later recurrence
Which lessons are mounted at a prompt	Token overlap, unmeasured	Recurrence before and after the date (intent design §7, Lessons)
What the router suggests to a session	Keywords, about 2% precision	Accepted against declined, once input is limited to the user's
The order of questions in a founder batch	Not built	Answers that changed a blocking set

Conditions on any learned choice:

1. It picks inside a set the frame computed. It never adds to the set or widens anything.
2. Its reward is the trust plane's mechanical, lagged labels (§7's signals, I6's attribution). It never grades itself. Attribution already flows only from trust to capability; that is Felix's stop-gradient.
3. Every choice is recorded: the admissible set, the pick, the probability it had, the policy version and the seed. The decision is replayable even though it is not deterministic: a stranger can re-derive what was chosen and why it was allowed.
4. Below a stated sample size the policy reads UNKNOWN and a fixed default decides.
5. "Better" is scored only on the sealed corpus or a held-out replay, never on the archive the policy learned from.
6. It is not the model minimizer. Picking the cheapest model above the floor is the host's (D1), so it is excluded.

This is LeCun's split again. The trust plane and the capability plane's binding boundary are the intrinsic cost. Learned choosing is the critic and the actor, which work against the cost and cannot touch it.

When. Not now. Learning needs outcomes, and Felix has two unattended merges and no outcome record. The router shows what learning from thin, one-sided data gives: 106 declines and about 2% precision. A reasonable earliest point is when the outcome record holds a few hundred lagged rows on two ecosystems.

The one thing to do early. When lane item 7 is specified, record two things for every choice: the options it had, and the probability the chosen one had. A deterministic policy logs probability 1 for its pick and 0 for everything else. Off-policy evaluation cannot score an option that had probability 0, so those logs can never say whether another choice would have done better. One field now is what makes learning possible later. Once there is volume, the same field allows occasional recorded exploration, in choosing decisions only.

8. When to look at JEPA again

- A latent world model for code or repositories is released with weights and shown to predict a diff's CI or test outcome. Then try it as a proposer against the rented model, on the sealed corpus.
- The outcome record holds thousands of lagged rows, enough to train and hold out a small predictor on frozen embeddings. Even then it may only rank or propose.

Neither of these moves anything in direction §5.

Sources

- LeCun, A Path Towards Autonomous Machine Intelligence (2022); the cost module as summarised in Dawid and LeCun, Introduction to Latent Variable Energy-Based Models (2023)
- Assran et al., V-JEPA 2 (2025) and Meta's post
- Balestriero and LeCun, LeJEPA (2025)
- Huang, LeCun and Balestriero, LLM-JEPA (ICLR 2026); limits read from the paper
- Terver et al., What Drives Success in Physical Planning with Joint-Embedding Predictive World Models? (2025)
- Meta FAIR, CWM (2025)
- AMI Labs: Sifted, AI Business
- awesome-jepa, for the 2026 spread

How this was made. One session. It read the intent design's §7, the capability plane thesis and direction, then searched the web. LLM-JEPA's limits were read from the paper itself. The other papers' figures come from abstracts, project pages and secondary summaries, and the 2026 spread from a curated list. The Felix figures were read on 10-19 from the land ledger and `git rev-list`. None was taken from memory.